# try, catch, finally and Control Flow

7 min read · Updated 28 Jul 2026 · Topic 28 of 65

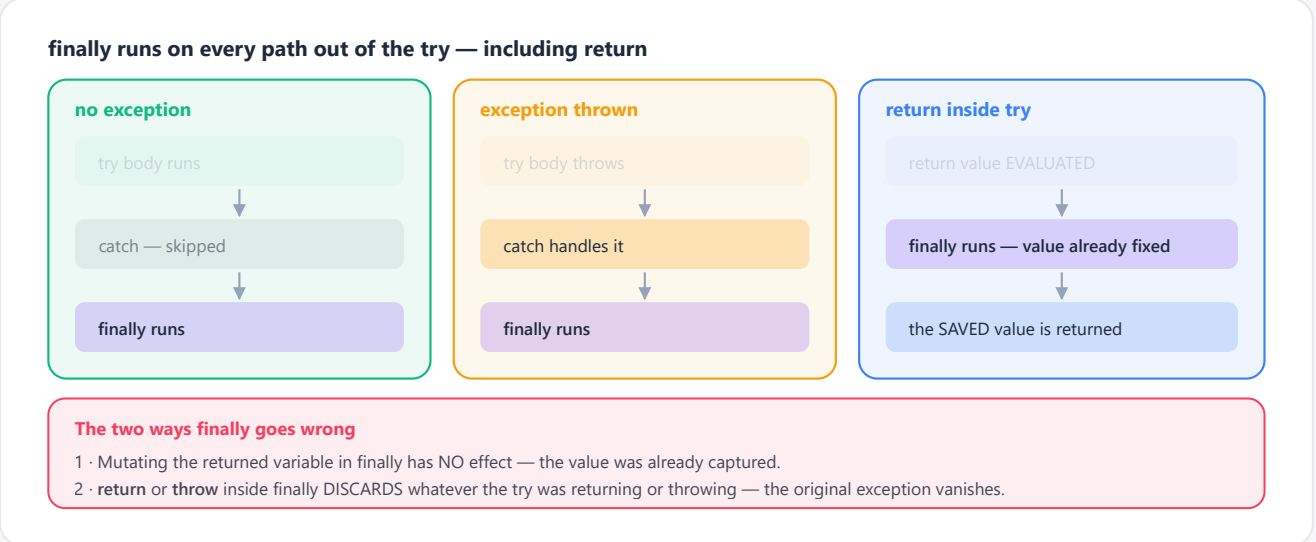
Bookmark · Mark done

## Why it matters

"What does this print?" questions built on `finally` are an interview staple, and they are not gotchas for their own sake — the rules they test explain two genuine production bugs: a value modified in `finally` that mysteriously does not change, and an exception that vanishes without trace. Both have caused real outages.

## The concept, simply

`finally` runs on **every** path out of the `try` block: normal completion, a caught exception, an uncaught exception on its way up, and `return`, `break` or `continue`. That is its entire purpose — guaranteed cleanup.




The third column is the one that produces interview questions: the return value is computed and saved BEFORE finally runs, so mutating the variable afterwards changes nothing.

## How it works

The rule that explains most puzzles: a `return` inside `try` evaluates its expression first, stores the result, then runs `finally`, then returns the stored result. Modifying the variable in `finally` is too late — the value has already been captured.

The two ways `finally` genuinely goes wrong:

- A `return` in `finally` **discards whatever the** `try` **was doing** — including an exception in flight. The exception simply disappears.
- A `throw` in `finally` **replaces the original exception**, so you debug the cleanup failure instead of the real one.

The only times `finally` does not run: `System.exit()`, the JVM crashing, or the thread being killed — all cases where the process is going away regardless.

## Code examples

The classic puzzle:

JAVA

Copy

```java
static int f() {
    int x = 1;
    try {
        return x;          // evaluates x → 1, SAVES it
    } finally {
        x = 2;             // too late — the return value is already 1
    }
}
System.out.println(f());   // 1

// With an object it looks different, and the rule is the same:
static List<String> g() {
    List<String> list = new ArrayList<>();
    try {
        return list;       // saves the REFERENCE
    } finally {
        list.add("x");     // mutating the object the saved reference poin
    }
}
System.out.println(g());   // [x]  — Chapter 1's pass-by-value, again
```

The exception-swallowing bug — this one is worth recognising instantly:

JAVA

Copy

```java
static int broken() {
    try {
        throw new RuntimeException("the real failure");
    } finally {
        return 42;          // ✗ DISCARDS the exception entirely
    }
}
System.out.println(broken());   // 42 — the exception vanished without a t

// Same class of bug, via a throw:
static void alsoBroken() throws IOException {
    try {
        throw new IOException("the real failure");
    } finally {
        cleanup();          // if cleanup() throws, IT propagates and the
    }                       // IOException is lost
}
// Rule: never return or throw from finally. Most linters flag it.
```

What `finally` is actually for, and the modern replacement:

JAVA

Copy

```java
// Legitimate use — releasing something that must be released.
lock.lock();
try {
    doWork();
} finally {
    lock.unlock();          // ✓ runs even if doWork() throws
}

// For anything AutoCloseable, prefer try-with-resources (next topic),
// which generates the finally correctly and handles suppressed exceptions
```

And `try` / `finally` with no `catch`, which is often the right shape:

```
// "I cannot handle this, but I must clean up." The exception propagates
// untouched, which is exactly right — no catch block needed.
void process() throws IOException {
    startTransaction();
    try {
        doWork();
    } finally {
        endTransaction();
    }
}
```

## Under the hood

`finally` has no dedicated bytecode instruction. The compiler **duplicates the `finally` block** onto every exit path — after the try body, inside each catch, and in a synthetic catch-all handler that rethrows. That is why a large `finally` block genuinely increases method size, and it explains the semantics exactly: since the block is copied after the return value has been computed, mutating the variable cannot affect it.

The historical footnote worth knowing is `jsr` / `ret` — the original bytecode instructions for subroutines, used for `finally` in early Java. They were so difficult for the verifier to reason about that they became a source of security bugs, and they were deprecated; modern `javac` uses duplication instead.

`return` **versus** `break` / `continue` follows the same rule: any control transfer out of the `try` triggers `finally` first. And nesting behaves as you would expect — an inner `finally` runs before an outer one as the exception unwinds outward.

## Common mistakes

- `return` **inside** `finally`**.** Silently discards exceptions. Never do it.
- **Throwing from** `finally`**.** Replaces the real exception with the cleanup failure.
- **Expecting a mutation in** `finally` **to change a returned primitive.** The value was captured first.

- **Closing resources by hand in `finally`.** Use try-with-resources — it gets the nesting and suppression right.
- **Assuming `finally` always runs.** `System.exit()` and a JVM crash skip it.
- **Heavy logic in `finally`.** It is duplicated onto every exit path and runs during exception unwinding. Keep it to cleanup.

## Interview lens

**Q: "When does `finally` not run?"**
A: When the JVM stops — `System.exit()`, a hard crash, or the thread being killed. Otherwise it runs on every path: normal completion, a caught exception, an exception propagating out, and `return`, `break` or `continue`.

**Q: "`int x = 1; try { return x; } finally { x = 2; }` — what is returned?"**
A: 1. The `return` expression is evaluated and the value stored before `finally` executes, so assigning to `x` afterwards has no effect on what was already captured. If the return were an object reference, mutating the object *would* be visible — because the saved reference still points at it — but reassigning the variable would not.

**Q: "What happens if you return from `finally`?"**
A: It overrides everything the `try` was doing, including an exception in flight — the exception is discarded entirely and the caller sees a normal return. It is one of the few genuinely silent failure modes in Java, and it is why linters flag any `return` or `throw` inside a `finally` block.

**Q: "Can you have `try` without `catch`?"**
A: Yes — `try`/`finally` with no `catch` is perfectly valid and often the right shape: it says "I cannot handle this failure, but I must clean up", and the exception

propagates untouched. You need at least one of `catch` or `finally`, or, since Java 7, a resource specification.

**Q: "How is `finally` implemented?"**

A: There is no bytecode instruction for it. The compiler duplicates the block onto every exit path — after the try body, inside each catch, and in a synthetic catch-all that rethrows. That duplication is exactly why the return value is fixed before the block runs. Early Java used `jsr` / `ret` subroutines instead, but they were hard to verify and were dropped.

## Cheat-card

- `finally` runs on **every** exit: normal, caught, uncaught, `return` / `break` / `continue`.
- Skipped only by `System.exit()`, JVM crash, thread kill.
- **Return value is evaluated and saved BEFORE `finally` runs.**
- Mutating the returned *object* is visible; reassigning the *variable* is not.
- **Never `return` or `throw` from `finally`** — it discards the real exception.
- `try` / `finally` with no `catch` is valid and often correct.
- Implemented by **duplicating the block** onto each exit path — keep it small.
- For closeable things, use **try-with-resources** instead.

QUICK CHECK — no marks, just for you.

1. int x = 1; try { return x; } finally { x = 2; } What is returned?

- ○ A compile error
- ○ Undefined behaviour
- ○ 1 — the return expression is evaluated and the value saved before finally runs

2 — finally runs last, so it wins

Check answer

2. What happens if you return from inside a finally block while an exception is propagating?

- ◯ Both the exception and the return value reach the caller
- ◯ The exception is rethrown after the return
- ◯ It is a compile error
- ◯ The exception is DISCARDED entirely and the caller sees a normal return

Check answer

3. When does a finally block NOT run?

- ◯ When System.exit() is called, the JVM crashes, or the thread is killed
- ◯ When the try block returns
- ◯ When an exception is thrown but not caught
- ◯ When a break or continue exits the try

Check answer

Java Interview Kit

LEARN MORE

## ARTICLES & DOCS

- The finally block — The Java Tutorials
  docs.oracle.com
- The try statement (JLS §14.20)
  docs.oracle.com

## VIDEOS




Try with Resources with Catch and Finally | Java Exception handling
Ram N Java

Hand-picked references for this lesson.